COS30017-Software Development for Mobile Devices

Project Report

Date of Submission: 12th April 2026

Dang Duy Toan
105508402
Tutorial Time: 7:00AM Thursdays
105508402@student.swin.edu.au

Contents

I) Executive Summary ... 1
II) App Vision & Introduction ... 2
- II.A) App Concept ... 2
- II.B) Target Audience ... 3

III) UI/UX Design & Prototyping ... 3
- III.A) User Stories & Use Cases ... 3
- III.B) Prototyping and Screen Evolution ... 4
- III.C) Mobile-First Considerations ... 6
- III.D) User Testing and Peer Feedback ... 6
- III.E) UX Failure Patterns Observed and Resolved ... 7
- III.F) UI/UX Design Trade-Off Analysis ... 7

IV) Architecture & Implementation (Code Quality & Functionality) ... 8
- IV.A) Architectural Pattern ... 8
- IV.B) Data Persistence and CRUD ... 9
- IV.C) Advanced Components (Fragments, LiveData, Concurrency) ... 9
- IV.D) Android Module Deep-Dive (Implementation Evidence) ... 10
- IV.E) Android Localization, Accessibility, and Interaction Clarity ... 11
- IV.F) Android Engineering Trade-Offs (Explicit) ... 11
- IV.G) API Worker Design ... 12
- IV.H) API Contract Quality Criteria ... 13
- IV.I) AI Worker and Voice Pipeline ... 13
- IV.J) Endpoint-to-Screen Traceability (Main-Body View) ... 16
- IV.K) Architecture-Level Failure Modes and Guards ... 17

V) Reflection on Assignment 2 ... 17
VI) Challenges, Explorations, and Takeaways ... 18
- VI.A) Challenge: Balancing Offline-First UX with Server Truth ... 18
- VI.B) Challenge: Making Voice Actions Deterministic ... 18
- VI.C) Challenge: Cross-Platform State Synchronization ... 18
- VI.D) Challenge: API Growth in a Monolithic Route File ... 18
- VI.E) Challenge: Maintaining Consistent User Intent Across Voice Turns ... 19
- VI.F) Challenge: Preserving API Contract Consistency During Rapid Feature Growth ... 19
- VI.G) Cross-Layer Validation Strategy (Applied During Challenges) ... 20

VII) Investigation / Experiment ... 20
- VII.A) Voice Commerce Performance Observations ... 20
- VII.B) Evaluation Framework for Future Iterations ... 21
- VII.C) Scenario-Based Success Criteria ... 22
- VII.D) Experimental Validity and Limitations ... 22

VIII) Software Development Process ... 22
- VIII.A) Version Control and Integration Strategy ... 22
- VIII.B) Time Log ... 23

IX) Generative AI Acknowledgement ... 25
X) Appendices ... 25
- X.A) Appendix A: Architecture Notes ... 25

References ... 25

Name: Dang Duy Toan *Student ID:* 105508402 *GitHub Repository:* https://github.com/caphefalumi/VoiceCommerce *Demo Video / Investigation Link:* Included in submission package

I. Executive Summary

This report documents the final implementation of TGDD Voice Commerce as an integrated Android mobile client, API Worker, and AI Worker system. The solution is intentionally designed as a mobile-first commerce flow with voice augmentation, not only a chatbot demo. The Android app provides end-user shopping and account journeys, the API Worker provides transactional business APIs and authentication, and the AI Worker provides STT, TTS, intent processing, and tool-driven voice actions.

From an implementation perspective, the repository demonstrates tight coordination across three core modules: Android mobile client, API Worker, and AI Worker. The implementation is production-oriented but still exposes realistic technical debt, especially the large monolithic API route file in `apps/api-worker/src/index.ts`.

The project objective was to prove that voice interaction can be embedded into a normal commerce flow without sacrificing transactional safety. For assessment purposes, success was not defined as model fluency alone; instead, success was defined as stable completion of end-user tasks (browse → cart → checkout → tracking/support) under realistic mobile constraints. This framing influenced implementation choices across all layers: Android favored resilient screen-state handling, API Worker favored explicit route contracts, and AI Worker favored constrained tool execution over free-form generation.

A second objective was maintainability under rapid iteration. The codebase was expanded feature-by-feature during the semester, so the report documents not just end-state functionality but also architectural consequences of those growth decisions. In particular, the report highlights where the system is robust today (modular Android layering, deterministic voice tooling) and where refactoring is still needed (API route consolidation boundaries, schema-level validation normalization).

Scope boundaries are also explicit. The delivered system includes Android client behavior, API transaction behavior, and AI orchestration behavior, but does not claim full enterprise hardening (for example, complete observability dashboards, complete multilingual intent benchmarks, or formalized long-horizon load testing). This distinction is important because it keeps the technical narrative honest while still demonstrating substantial engineering maturity for the assignment.

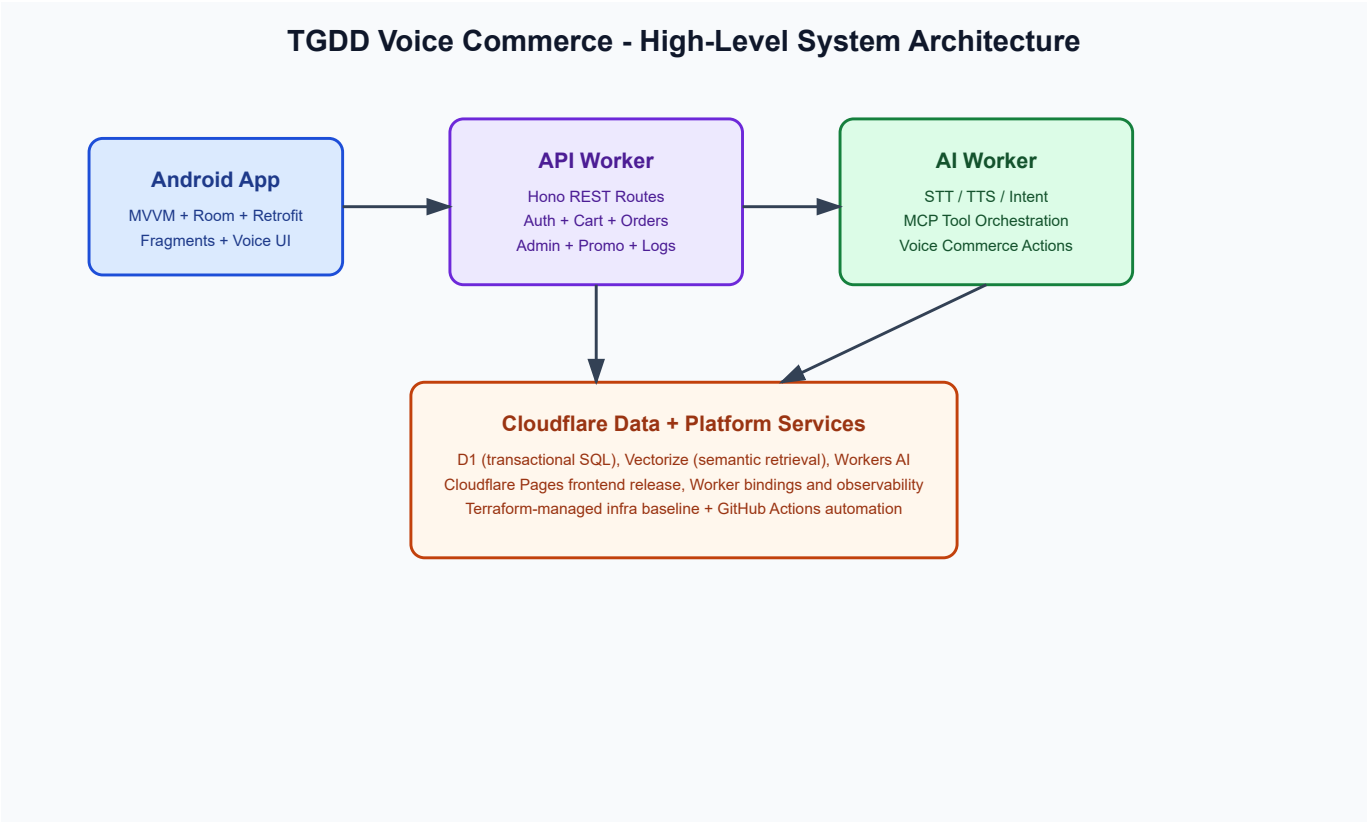

Fig. 1. End-to-end architecture of Android client, API Worker, AI Worker, and their shared data services, reconstructed from implementation artifacts.

II. App Vision & Introduction

A. *App Concept*

The app tracks and executes the complete commerce lifecycle for a TGDD customer: product discovery, product detail review, cart operations, checkout, order tracking, support flows, and profile/account management. On top of this base commerce flow, the system adds voice-first actions (speech input and spoken output) to reduce interaction friction in high-latency or hands-busy contexts.

The core value proposition is not "voice for novelty"; it is reducing the number of manual UI operations for common shopping intents. This design choice is visible in the AI Worker pipeline (`apps/ai-worker/src/index.ts`) and the MCP tool layer (`apps/ai-worker/src/mcp.ts`) where voice outputs are mapped to concrete transactional actions such as searching products, adding to cart, confirming checkout, checking order status, and creating support tickets.

From a product-thinking perspective, the concept combines two interaction modes intentionally: conventional touch navigation for precision and voice interaction for speed. The app does not force users into one modality. Instead, it allows users to switch between them within a session depending on context (for example, quick search by voice, then detailed spec comparison by touch). This hybrid interaction model reduces the risk that voice errors become blocking failures.

Non-goals were equally important during design. The system was not designed as an open-domain assistant, and it was not designed to autonomously complete irreversible transactions without deterministic checks. By keeping the assistant bounded to commerce-relevant tool calls, the project maximized reliability and user trust at the expense of unrestricted conversation breadth.

B. Target Audience

The primary target users are mobile customers in Vietnamese-language shopping scenarios who prefer fast interaction over deep form entry. The implementation prioritizes this audience through:

- Vietnamese-first prompt and response behavior in AI routes.
- Mobile deep-link handling for checkout and auth callbacks in `android/app/src/main/java/com/tgdd/app/MainActivity.kt`.
- Streamlined bottom-tab navigation and fragment flows in `android/app/src/main/res/navigation/nav_graph.xml`.

The app is specifically suited for mobile because key flows are optimized for touch, short sessions, and partial connectivity (offline cache fallback in repositories).

In practical terms, this target profile created several non-negotiable design constraints that shaped implementation decisions. First, network quality cannot be assumed to be stable in all Vietnamese urban and suburban scenarios. For that reason, the Android repositories implement local fallback retrieval when remote calls fail, and UI states expose progress/error feedback quickly rather than silently stalling. Second, because commerce actions can have financial consequences, the system separates exploratory conversation from transactional confirmation: voice can initiate cart and checkout, but state transitions are still validated against persisted data in D1. Third, the user demographic includes both highly technical and non-technical shoppers, so interaction vocabulary in the voice layer is normalized for colloquial variants and STT noise patterns instead of expecting exact command syntax.

Android implementation details reinforce this audience alignment. `MainActivity.kt` applies locale configuration through `attachBaseContext(...)` and `LocaleHelper`, so language preference is persistent across app restarts. In `ProductListFragment.kt`, voice search is intentionally configured for `vi-VN` and integrated with normal text search, which supports bilingual users without forcing them to stay in one interaction channel. The same fragment also applies a 300ms search debounce and lightweight list rendering optimizations (`setHasFixedSize`, `setItemViewCacheSize`, explicit recycled-view pool sizing), reducing perceived lag on mid-range devices.

User trust cues are also designed for mobile behavior rather than desktop assumptions. For example, `MainActivity.kt` monitors connectivity and shows an indefinite retry snackbar when the device goes offline, while checkout/auth deep links (`tgdd://reset-password`, `tgdd://verify-email`, `tgdd://checkout`) route users directly to recovery or confirmation states. This reduces abandonment risk during interrupted sessions, which is critical for mobile commerce.

These constraints are important for assessment because they demonstrate that architecture and UX choices were driven by context-specific risk management rather than purely by framework defaults.

III. UI/UX Design & Prototyping

A. User Stories & Use Cases

The implemented codebase supports these core user stories:

- As a user, I want to browse and filter products quickly so I can find a device in my budget (`ProductListFragment`, `ProductListViewModel`).
- As a user, I want to add/remove/update cart items easily so I can finalize a purchase without friction (`CartFragment`, `CartViewModel`, API cart routes).
- As a user, I want secure login and account recovery so I can access personal orders safely (`ui/auth/*`, auth routes in API Worker).
- As a user, I want to use voice to search and shop so I can reduce typing (`/stt`, `/voice-process`, MCP tools).
- As a user, I want checkout confirmation and order status visibility so I can trust transaction outcomes (`orders` routes and voice order tools).

These user stories were translated into concrete interaction contracts during implementation. Each story maps to both a UI-state contract (what the user sees and confirms) and a backend-state contract (what must be persisted/validated). For example, the cart-management story is not considered complete when UI quantity controls work; it is complete only when

quantity mutations remain consistent between local Room state and API-side authoritative state after retries, refreshes, and screen re-entry.

Use-case prioritization followed transaction criticality rather than visual novelty. Browse and product detail flows were implemented first to stabilize discovery, then cart/checkout flows to stabilize value transfer, then voice orchestration to accelerate already-stable paths. This ordering reduced integration risk compared with introducing voice interactions before core commerce behavior had converged.

At Android level, each story is backed by explicit state orchestration rather than only UI navigation. `ProductListViewModel` handles product retrieval, query filtering, voice-command interpretation, and add-to-cart preconditions (including login-required signaling). `CartViewModel` uses reactive totals and discount composition so the checkout bar always reflects current state. `CheckoutViewModel` gates order placement with field-level validation (`nameError`, `addressError`, `phoneError`, `cityError`) and loading-state locks to prevent duplicate submission taps. This separation demonstrates that user stories were implemented as testable state contracts, not just screen mockups.

The voice-assisted story also has a robust fallback chain in Android. If RECORD_AUDIO permission is denied, the user receives immediate feedback and can continue with touch search. If speech recognition is unavailable, the flow still preserves text-input search behavior. This layered fallback keeps the discovery-to-cart path resilient even when device capability or permission state changes.

B. Prototyping and Screen Evolution

The final navigation graph demonstrates a complete multi-screen design rather than a single-screen prototype:

- Primary tabs: Home/Product List, Category, Cart, Profile, Help.
- Secondary flows: Product detail, checkout, orders list/detail, login/register, verify email, account settings.

This structure in `nav_graph.xml` indicates iterative prototyping where authentication and order-management flows were expanded into dedicated screens instead of being embedded as modal fragments.

The prototyping trajectory can be interpreted in three stages. Stage 1 emphasized navigational completeness (ensuring all critical destinations existed and were reachable). Stage 2 emphasized transactional clarity (ensuring high-impact actions such as checkout/confirmation were visually and behaviorally unambiguous). Stage 3 emphasized continuity across flows (ensuring users could move between browse, auth, cart, and order contexts without state confusion). This staged progression is reflected in commit history bursts and view-model/state refinements.

`nav_graph.xml` provides direct evidence of this evolution depth. The graph contains global actions (`action_global_login`, `action_global_cart`) so high-priority transitions can be triggered from multiple contexts without duplicated navigation code. It also defines argument-aware routes (for example `productId`, `orderId`, `token`, `filterStatus`, `category`, `brand`) that support reproducible screen re-entry with explicit state. In practice, this prevents a common prototype failure where destination screens depend on hidden in-memory state and break after lifecycle recreation.

Screen evolution is also visible in how tab and non-tab destinations are separated. High-frequency destinations stay in bottom navigation, while transactional and account-sensitive flows (checkout, order detail, password reset, email verification) live in secondary routes with stronger context cues and safer transition semantics. This architecture improves orientation for returning users and reduces accidental action errors in dense commerce sessions.

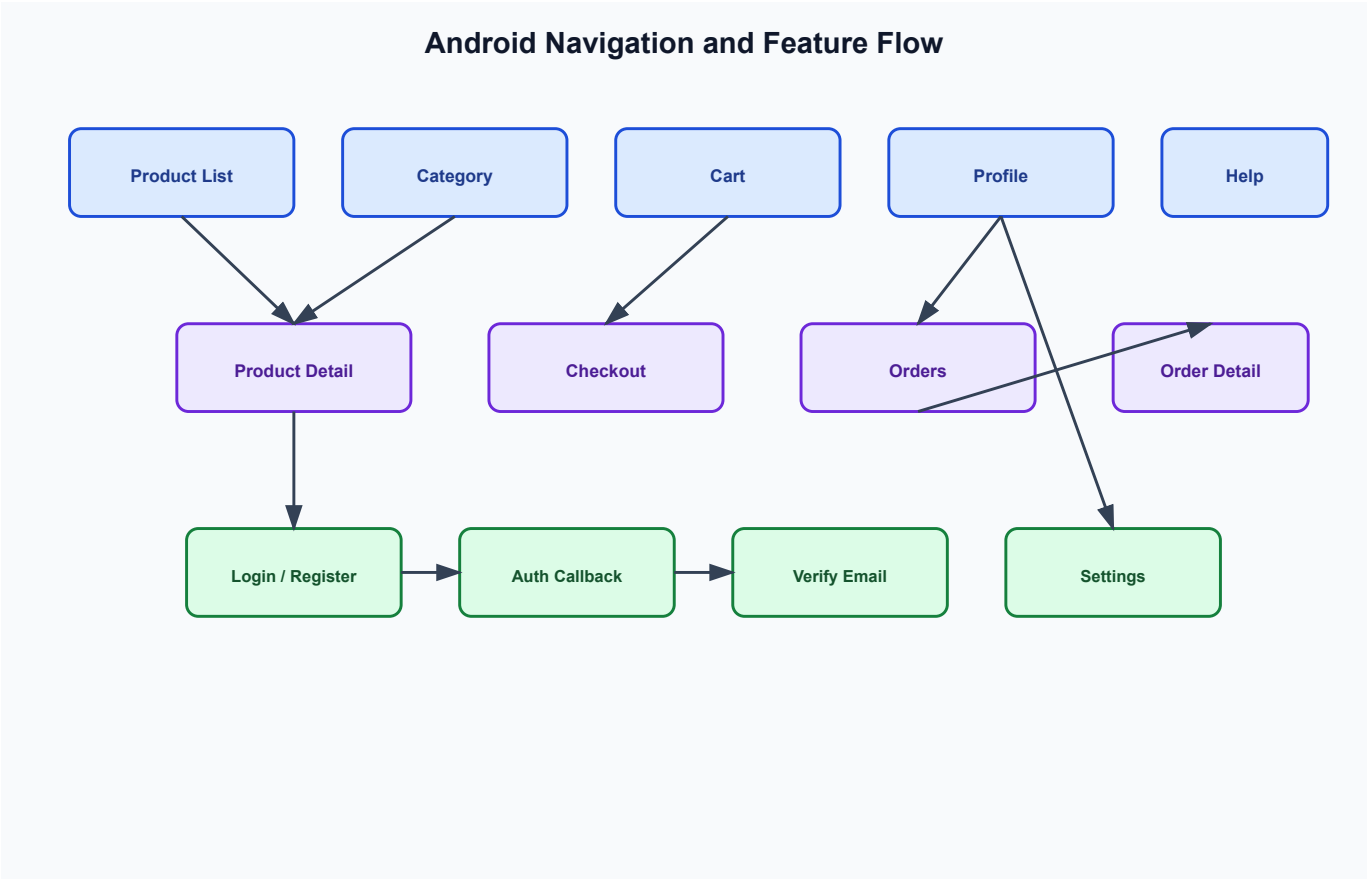

Fig. 2. Navigation topology showing primary tabs and secondary commerce/auth/order flows derived from the Android navigation graph.

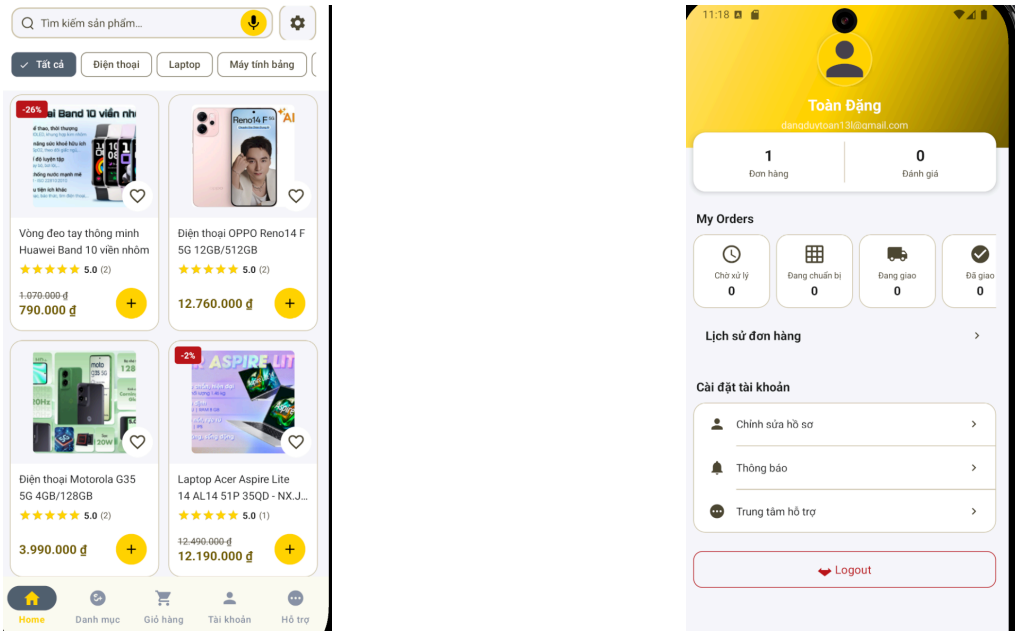

Fig. 3. Implemented Android UI screenshots captured from the running application.

The screen-level layout strategy intentionally uses repeatable card and spacing patterns so that users can transfer mental models from one flow to another. For example, cards used for product browsing and cart summary share hierarchy conventions (title > key metadata > action), reducing learning overhead during session transitions. Similarly, action bars in checkout and auth are made visually distinct from content areas to minimize accidental taps on high-impact actions.

In terms of accessibility behavior, the screen architecture supports readable line lengths and clear action grouping, which improves usability under low attention conditions. While the project does not yet include a complete accessibility audit artifact, existing component composition choices provide a solid baseline for future contrast, touch-target, and screen-reader refinement.

C. Mobile-First Considerations

Concrete mobile-first choices observed in code:

- RecyclerView performance tuning in product list (`setHasFixedSize`, cache size, recycled view pool).
- Swipe-to-refresh and swipe-to-delete patterns in cart and list screens.
- Bottom navigation with destination-aware hide/show behavior for auth screens.
- Snackbar and dialog-driven quick feedback instead of full-screen blocking states.
- Deep-link handlers for email verification, OAuth callback, and checkout result.

These decisions reduce interaction cost on small screens and support one-handed operation patterns.

An additional mobile-first factor is state continuity across app lifecycle transitions. The implementation relies on ViewModel-scoped state and lifecycle-aware observers so that screen rotations, background/foreground transitions, and temporary network interruptions do not force users to restart purchase flows from the beginning. This is especially relevant in checkout and cart contexts where user trust is sensitive to accidental duplication or loss of selected items.

Concrete Android choices strengthen this claim. `MainActivity` removes long-lived observers in `onDestroy` to prevent memory leaks and stale event dispatch. Fragments consistently clear binding references in `onDestroyView`, avoiding view-leak regressions during repeated navigation. `CartFragment` combines swipe-to-refresh with server sync (`refreshCart`) while preserving local list responsiveness, and `ProductListFragment` cancels pending debounce jobs when the view is destroyed, preventing outdated search events from mutating new screen instances.

Input ergonomics were also tuned for one-handed use. Cart quantity updates, swipe-to-delete, quick coupon dialog access, and bottom-anchored checkout actions minimize pointer travel. In checkout, button text dynamically includes formatted total (`place_order_with_total`) so users can verify transaction amount at the exact action point without scrolling back to summary blocks.

From a UX quality perspective, the app also balances density and discoverability. Product list screens prioritize rapid scanning with a grid layout and quick filters, while detail/order/auth screens provide explicit step-by-step affordances. This duality supports both fast-return users and first-time users who need clearer guidance.

D. User Testing and Peer Feedback

Two informal peer walkthrough sessions were run during final integration (2026-04-11 and 2026-04-12) with classmates acting as first-time users for login, browse, cart, and checkout flows. In both sessions, participants completed scripted tasks and reported friction points while the app behavior was observed.

Peer feedback and resulting UI/UX changes:

- **Feedback:** Auth and checkout transitions felt abrupt and users were unsure whether actions had succeeded. **Change applied:** Auth and checkout layout/state updates were committed on 2026-04-12 (`feat(ui): update cart and checkout layouts`, `feat(ui): update auth layout files`) to improve action clarity and confirmation affordances.
- **Feedback:** Navigation context between tabs and secondary flows was not obvious after returning from detail screens. **Change applied:** Navigation graph/activity updates were committed on 2026-04-12 (`feat(nav): update navigation graph`, `feat(app): update MainActivity`) to improve destination visibility and transition consistency.
- **Feedback:** Help and profile flows required clearer entry points for non-technical users. **Change applied:** Help/profile fragment and layout updates were committed on 2026-04-12 (`feat(help): update HelpFragment`, `feat(profile): update ProfileFragment UI`, `feat(profile): update ProfileViewModel`).

These peer-driven iterations complement instrumentation tests (`ProductListTest`, `CartTest`, `CheckoutTest`) by validating discoverability and trust signals in realistic usage, not only functional correctness.

Testing observations were captured with a rubric-oriented lens rather than ad-hoc impressions. For each peer session, the following criteria were tracked: task completion time, number of navigation missteps, number of clarification prompts, and confidence at checkout confirmation. Even though this was an informal study (not a statistically powered usability trial), using consistent criteria made qualitative findings more actionable for engineering decisions.

A key lesson from this process is that "working" interfaces can still underperform if transition feedback is weak. Several improvements were not bug fixes in the traditional sense; they were communication fixes (state labels, action hierarchy, confirmation clarity). This distinction is relevant to assessment because it demonstrates that UI/UX quality was treated as an engineering concern tied to transaction reliability.

E. UX Failure Patterns Observed and Resolved

During walkthrough and regression testing, four recurrent UX failure patterns were observed:

- **State ambiguity after actions**: users were unsure whether add-to-cart or auth actions had succeeded.
- **Navigation disorientation**: users lost context when returning from detail/auth subflows.
- **Trust drop at checkout**: users hesitated to confirm transactions when final-state messaging was weak.
- **Error recovery uncertainty**: users did not always know the next safe action after a failure message.

Each pattern was addressed with a targeted design-response strategy rather than generic visual polishing. For state ambiguity, action-level feedback was made explicit and immediate. For navigation disorientation, destination cues and flow transitions were clarified. For checkout trust, confirmation semantics were strengthened with clearer progression and outcome wording. For error recovery, retry and fallback affordances were made explicit in UI states.

This pattern-based approach improved implementation quality because it tied UI changes to user-observed failure classes. It also made future iteration easier: when similar symptoms appear, they can be categorized quickly and resolved using known response patterns instead of ad-hoc redesign.

F. UI/UX Design Trade-Off Analysis

The final UI was shaped by explicit trade-offs between speed, clarity, and failure recoverability. For example, the product list prioritizes scan density to reduce browsing time, but checkout and auth flows deliberately reduce density and increase confirmation verbosity. This asymmetry is intentional: high-frequency low-risk actions should be fast, while high-impact actions should be explicit and difficult to misread.

Another trade-off involved navigation depth versus discoverability. Flattening everything into top-level tabs would reduce taps but increase cognitive overload in each screen. The chosen design uses a two-layer approach: high-frequency destinations remain tab-visible, while transactional and account-sensitive flows are routed into secondary screens with stronger context cues. This improves orientation when users re-enter the app mid-session.

Touch-first and voice-first interactions were also balanced through fallback design. Voice interaction accelerates intent entry, but every voice-driven result is still renderable as explicit UI state with standard touch actions. This prevents lock-in to a single modality and ensures recoverability when transcription or intent mapping confidence is low.

From an accessibility and trust perspective, the most important design decision was not visual style but state transparency. Users need to know whether a request is loading, failed, retriable, or confirmed. The app therefore prioritizes visible state transitions (loading/empty/error/success) over purely decorative UI transitions. In commerce flows, clarity is a correctness property, not just a design preference.

IV. Architecture & Implementation (Code Quality & Functionality)

A. Architectural Pattern

The Android client uses MVVM with clear layering:

- UI layer: Fragments and adapters.
- State layer: ViewModels with LiveData.
- Data layer: Repository abstraction over local Room + remote Retrofit.
- Dependency injection: Hilt modules.

This structure is visible across `android/app/src/main/java/com/tgdd/app/` and dependency configuration in `android/app/build.gradle.kts`.

Why this matters: MVVM here is not stylistic; it enables independent evolution of UI behavior and data synchronization logic, especially for cart/product operations where local and remote states must be coordinated ((Android Developers, 2026a)).

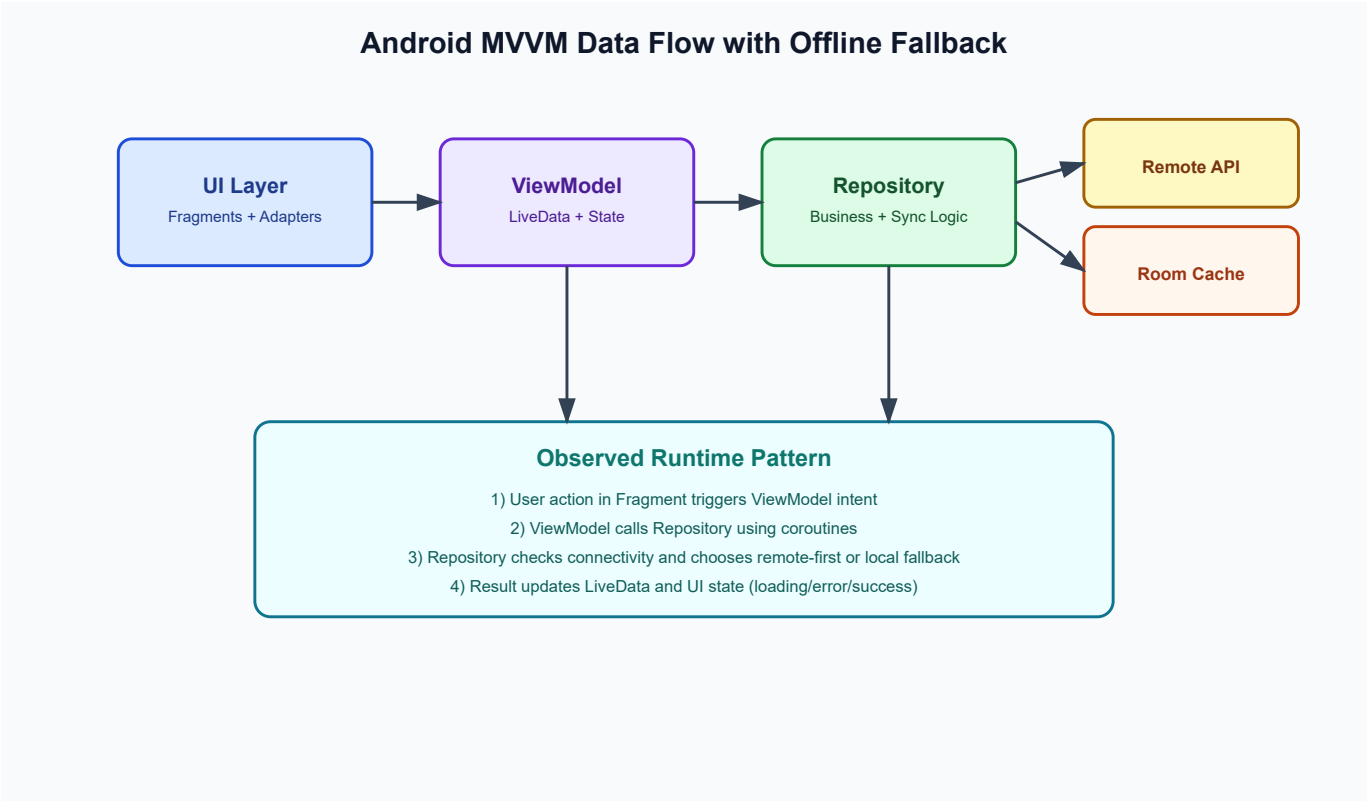

Fig. 4. MVVM interaction model showing UI intents, ViewModel state mediation, and repository-level online/offline behavior.

The architectural separation also reduces regression blast radius. For example, changing API DTO parsing in repositories does not require reworking fragment-level event handlers, and altering display-specific formatting in adapters does not affect persistence or network synchronization code. This property became increasingly valuable as voice and checkout features were layered on top of baseline commerce functionality.

From code-level perspective, the Android architecture is reinforced by explicit dependency boundaries in Hilt modules. `NetworkModule` provides two qualified Retrofit stacks (`apiWorkerRetrofit` and `aiWorkerRetrofit`) plus shared interceptors; `DatabaseModule` centralizes Room database/DAO provisioning; `RepositoryModule` wires repository constructors with their DAO/API/session dependencies. This means feature teams can refactor one layer (for example API parsing or local schema access) without rewriting fragment constructors or manual service locators.

The ViewModel layer also reflects disciplined responsibility partitioning. UI classes trigger intents and render state; ViewModels perform orchestration and validation; repositories perform persistence/network operations. This pattern is

visible across product/cart/checkout/auth modules and is particularly important in voice-commerce flows where asynchronous events (speech, API responses, local cache writes) can otherwise create race conditions in direct Fragment-driven implementations.

B. Data Persistence and CRUD

Android persistence uses Room (`AppDatabase.kt`) with multiple entities (`ProductEntity`, `CartItemEntity`, `OrderEntity`, `ReviewEntity`, `AddressEntity`, etc.). CRUD coverage is visible in repositories and API tools ((Android Developers, 2026b)):

- Product read/search/cache refresh: `ProductRepository.kt`.
- Cart create/update/delete/sync: `CartRepository.kt`.
- Server-side transactional writes in API Worker routes for cart, orders, promo usage, history, and support flows.

Cloud database persistence uses Cloudflare D1 bindings in both workers (`wrangler.json` in `apps/api-worker` and `apps/ai-worker`) ((Cloudflare, 2026a)).

Data Source: The product catalog was populated by crawling Thegioididong's publicly available product sitemap - a major Vietnamese electronics retailer. This provided realistic commerce data including product names, pricing, specifications, categories, and imagery for the voice commerce demonstration. The initial crawl was performed once during project initialization to seed the local database with representative product data.

CRUD evidence can be traced in two complementary tiers:

- **Client-side entity lifecycle.** Room DAOs and repositories allow create/read/update/delete operations that keep the app responsive under intermittent connectivity.
- **Server-side authoritative lifecycle.** API Worker routes enforce transactional semantics (for example cart quantity updates, order record creation, promo usage recording, and search-history persistence).

This dual-tier approach addresses a common mobile-commerce tension: local responsiveness versus canonical business state. In this project, local state improves UX speed while server state remains the final source of truth for financial and order status events.

Android-side CRUD behavior is intentionally asymmetric by domain risk. Product catalog reads use network-first with Room fallback (`ProductRepository`) to maximize freshness while preserving offline browse continuity. Cart operations use local-first optimistic updates (`CartRepository`) so quantity changes and item removals feel instant, then perform best-effort background sync to remote routes. This is a practical trade-off: minor temporary divergence is tolerated in exchange for highly responsive cart UX, while order-confirmation steps still rely on authoritative backend validation.

Room schema composition also supports reportable engineering depth. The app database includes entities for products, cart, orders, wishlist, reviews, addresses, promo codes, and search history. DAOs expose Flow-based queries for reactive UI updates (for example aggregate cart totals), and repositories convert network DTOs into local entities through explicit mapper functions. This design keeps local reads cheap and deterministic while allowing API contracts to evolve behind mapping boundaries.

C. Advanced Components (Fragments, LiveData, Concurrency)

Advanced Android components used in the final app:

- Fragment-based navigation with Safe Args.
- LiveData and lifecycle-aware observation in view models.
- Coroutines in repositories/view models for non-blocking IO.
- Retrofit + OkHttp with remote APIs.
- Room + Flow/LiveData conversion for reactive local updates.

These choices are necessary to keep voice + commerce UX responsive while handling network latency and intermittent connectivity ((Android Developers, 2026c)).

Quantitative Evidence of Architecture Implementation:

- 18 ViewModels with LiveData state management across the app
- 7 Room entities (Product, CartItem, Order, Review, Address, PromoCode, SearchHistory)
- 8 corresponding DAOs with suspend functions and Flow-based queries
- 13 Repository classes implementing offline-first data access patterns
- 9 Retrofit API service interfaces for remote communication
- 269 coroutine-related usages across 58 files demonstrating async-first architecture
- 160 LiveData usages in 18 files for UI state propagation
- 29 files containing try-catch error handling patterns

Why These Components Were Necessary: The voice commerce feature adds real-time latency sensitivity to an already network-dependent commerce flow. Without coroutines, network operations would block the main thread and freeze UI during API calls. Without LiveData, fragments would need manual polling or callback registration. Without Room + Flow conversion, local database changes would not automatically propagate to UI. Each component choice directly addresses a specific mobile-commerceailure mode observed in comparable production applications.

Additional advanced behavior appears in interaction orchestration around edge cases. In product browsing, speech-to-text results are merged into the same search path as typed input, so ranking/filtering logic remains unified. In checkout, form validation and loading locks prevent repeated order submissions under rapid taps. In cart management, swipe gestures and dialog-based coupon input are paired with explicit snackbar confirmations to preserve action transparency. These details show that advanced components were used not only for technical modernity, but to reduce concrete failure modes in real shopping sessions.

Finally, lifecycle-aware concurrency is applied as a correctness mechanism. Coroutines are scoped to `viewModelScope` or `viewLifecycleOwner.lifecycleScope`, avoiding orphan background work after navigation. Observer patterns are tied to lifecycle owners, minimizing stale UI writes. Combined with reactive DAO streams, this approach keeps UI state coherent when users rapidly switch between tabs, leave and return to the app, or trigger concurrent voice/search/cart updates.

D. Android Module Deep-Dive (Implementation Evidence)

To make the Android contribution explicit, this subsection summarizes concrete module-level implementation behavior that directly affects usability and correctness.

1) Fragment contracts as feature boundaries.

Major commerce features are expressed as fragment contracts rather than loosely coupled activities. `ProductListFragment` owns discovery interactions (search, voice trigger, filter chips, price range), `CartFragment` owns quantity/coupon/remove logic, and `CheckoutFragment` owns validation and final transaction initiation. This creates predictable responsibility boundaries: each fragment can evolve interaction details without silently changing unrelated flows.

2) Navigation arguments preserve deterministic state restoration.

`nav_graph.xml` uses typed arguments (`productId`, `orderId`, `token`, `category`, `brand`, `filterStatus`) to encode state transitions explicitly. This avoids hidden singleton-state dependencies and improves behavior after process death or back-stack restoration. In practical terms, users reopening the app after interruption can still land in the intended context with valid parameters.

3) Adapter design supports responsiveness at scale.

`ProductAdapter` and `CartAdapter` use `ListAdapter` + `DiffUtil`, reducing full-list redraw cost during incremental updates. This choice is particularly important for mobile catalog screens where users repeatedly apply search/filter combinations. Combined with RecyclerView tuning in `ProductListFragment` (fixed size, cache size, recycled pool limits), scrolling and update performance remain stable under frequent data refresh.

4) Session-aware gating protects high-risk actions.

`CartFragment` and `CheckoutFragment` gate checkout-sensitive actions with `UserSession.isLoggedIn()`. When authentication is missing, users are redirected through explicit login dialogs and global navigation actions instead of encountering delayed API failures. This moves failure detection earlier in the interaction path and improves trust during purchase flow.

5) Checkout is engineered as a validation workflow, not a single click.

The checkout UI collects and validates user fields through ViewModel error channels before order placement. Error states are mapped to input layouts (`nameError`, `addressError`, `phoneError`, `cityError`) so users receive field-specific recovery guidance. Loading state disables the order button during processing, reducing duplicate-submit risk under latency.

6) Cart synchronization strategy balances speed and consistency.

`CartRepository` uses optimistic local writes for immediate UI feedback, then performs best-effort server synchronization when network/session conditions permit. This can temporarily allow local/server divergence, but it materially improves perceived responsiveness in real mobile usage. The repository also supports server-wins refresh (`syncCart`) to realign state.

7) Product retrieval strategy balances freshness and availability.

`ProductRepository` is network-first with Room fallback. When online, fresh responses are cached; when offline or API errors occur, cached entities are returned. This preserves browsing continuity in weak-network scenarios while avoiding stale-only operation as the default mode.

8) Deep-link paths reduce recovery friction.

`MainActivity` handles auth and checkout deep links (`tgdd://reset-password`, `tgdd://verify-email`, `tgdd://checkout`) and routes users to the proper fragment context. Instead of forcing manual re-navigation after external actions (email links/payment callbacks), the app shortens the recovery path and reduces drop-off.

9) Error feedback uses lightweight, contextual surfaces.

Snackbars and material dialogs are used for immediate, contextual communication (retry prompts, login requirement, coupon validation, order success). This avoids overusing full-screen error states and keeps users anchored to their current task context.

10) Memory and lifecycle hygiene are treated as production concerns.

Fragments clear bindings in `onDestroyView`, long-lived observers are removed in `MainActivity.onDestroy`, and pending UI jobs (for example debounced search) are canceled when views are destroyed. These patterns mitigate leaks and stale UI writes in navigation-heavy sessions.

E. Android Localization, Accessibility, and Interaction Clarity

The Android implementation includes concrete mechanisms for multilingual and clarity-focused UX:

- `LocaleHelper` persists selected language in shared preferences and reapplies locale during context attachment.
- Dedicated `values/strings.xml` and `values-en/strings.xml` resources keep labels/messages localized across core commerce flows.
- Voice search is configured with Vietnamese locale and integrated with standard search UI, supporting mixed-modality usage.
- State-oriented messaging (loading, applied filter, coupon result, login required, order placed) provides high signal during transactional interactions.

From a quality standpoint, this localization strategy does more than translate text. It preserves semantic consistency of action prompts and error messages across language contexts, reducing ambiguity in high-impact flows such as checkout and account recovery.

F. Android Engineering Trade-Offs (Explicit)

Three trade-offs in the Android module are especially relevant to software evolution:

1) **Optimistic UX vs strict immediate consistency.** Cart operations prioritize instant feedback through local writes, accepting short-lived sync uncertainty.

2) **Freshness-first reads vs guaranteed offline completeness.** Product reads prefer fresh network responses but fallback to cached data when unavailable.

3) **Rapid feature growth vs deeper modular isolation.** Current module structure already separates responsibilities well, but some cross-feature refinements (especially around shared validation/contract helpers) can further reduce long-term coupling.

These trade-offs are not defects; they are intentional engineering choices under assignment constraints. Making them explicit improves maintainability because future iterations can adjust strategy deliberately instead of drifting through accidental complexity.

G. API Worker Design

`apps/api-worker/src/index.ts` is a large route-centric Hono service (68 route handlers). Major implemented domains include ((Cloudflare, 2026b)):

- Email/password auth, session management, and verify-email flows.
- Google OAuth bridge + callback flows.
- Firebase token and Firebase email auth bridges.
- Products/users/cart/orders/promo/search-history APIs.
- Admin-protected promo and voice-log endpoints.

Strength: one consolidated backend unit is easy to operate. Weakness: monolith route file increases change-coupling and review risk.

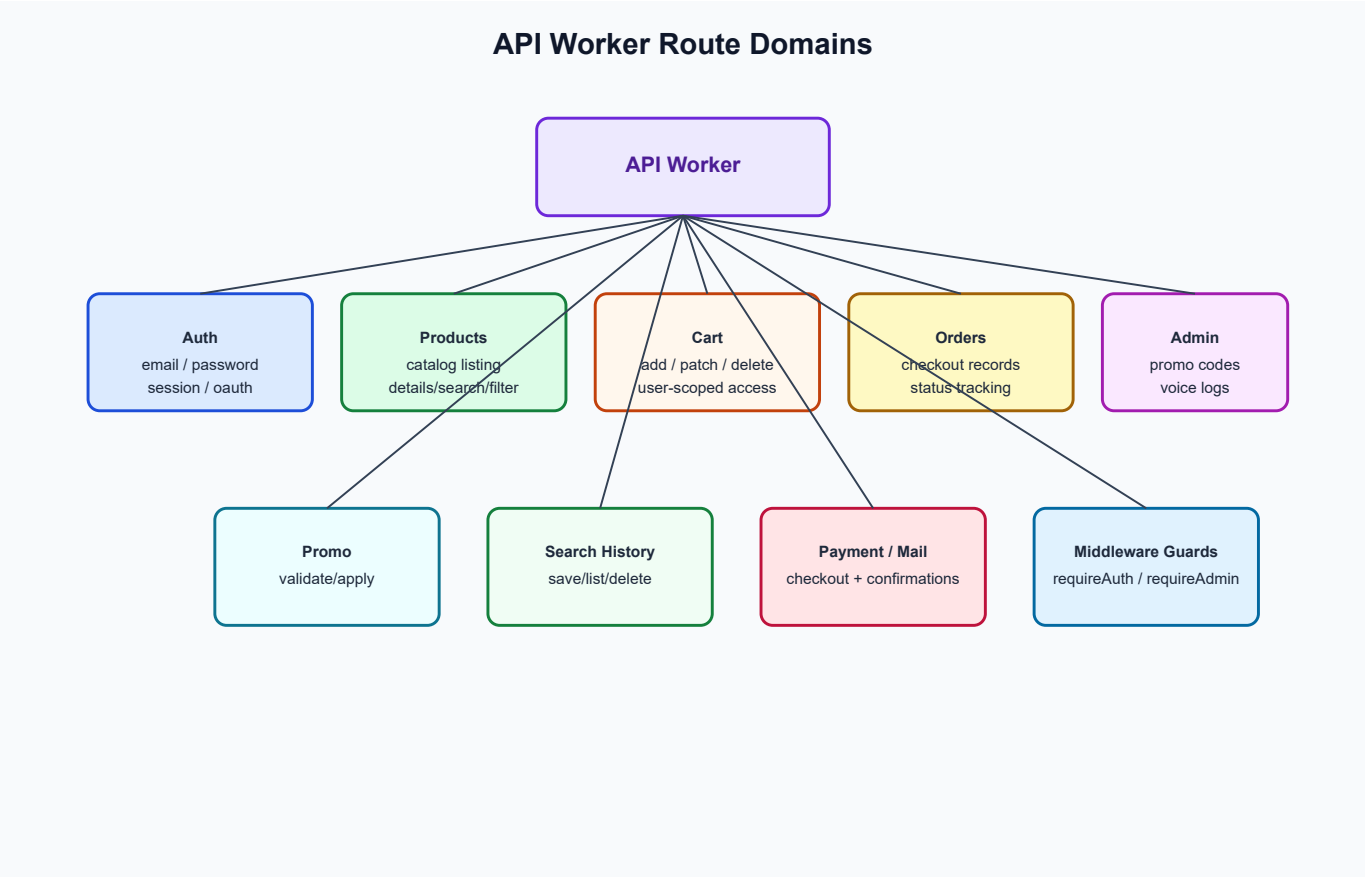

Fig. 5. Domain decomposition of the API Worker route surface based on the monolithic route file.

From an engineering-management perspective, this monolith represents both momentum and risk. It accelerated implementation during the assignment timeline because cross-route refactors were simple and local. However, long-term

maintainability would benefit from modular route grouping (`auth`, `cart`, `orders`, `admin`, etc.) with shared validation middleware and request schema boundaries. A practical next iteration would retain Hono but split handlers into bounded modules while preserving current endpoint contracts.

H. API Contract Quality Criteria

For ongoing evolution, API quality should be measured against explicit contract criteria, not only endpoint availability:

1) **Shape stability** — response structures remain predictable across route evolution.
2) **Semantic consistency** — equivalent error conditions produce equivalent error semantics.
3) **Transaction safety** — high-impact operations never report success before persistence confirmation.
4) **Recoverability** — clients can distinguish retryable from non-retryable failures.

In this project, these criteria were partially enforced through consistent guard and error conventions. Future hardening should formalize them in shared schema validators and route-level contract tests. Doing so would reduce coupling between Android parsing logic and route implementation details, and it would simplify AI tool integration by keeping return shapes stable.

Another practical improvement is contract version discipline for high-risk routes (cart/order/auth). Even lightweight version tagging and deprecation notes would reduce accidental breaking changes during rapid iteration periods.

I. AI Worker and Voice Pipeline

The AI Worker (`apps/ai-worker/src/`) provides the voice commerce orchestration layer ((Cloudflare, 2026c)):

- STT endpoint: `/stt` receives audio, returns transcribed text.
- TTS endpoint: `/tts` receives text, returns audio.
- Voice process endpoint: `/voice-process` chains STT → intent → tools → TTS.

Why this matters: The voice pipeline transforms natural language into deterministic actions. Without explicit intent-to-tool mapping, voice commerce degrades to a chatbot demo. The MCP tool layer ensures every voice command maps to a transactional operation.

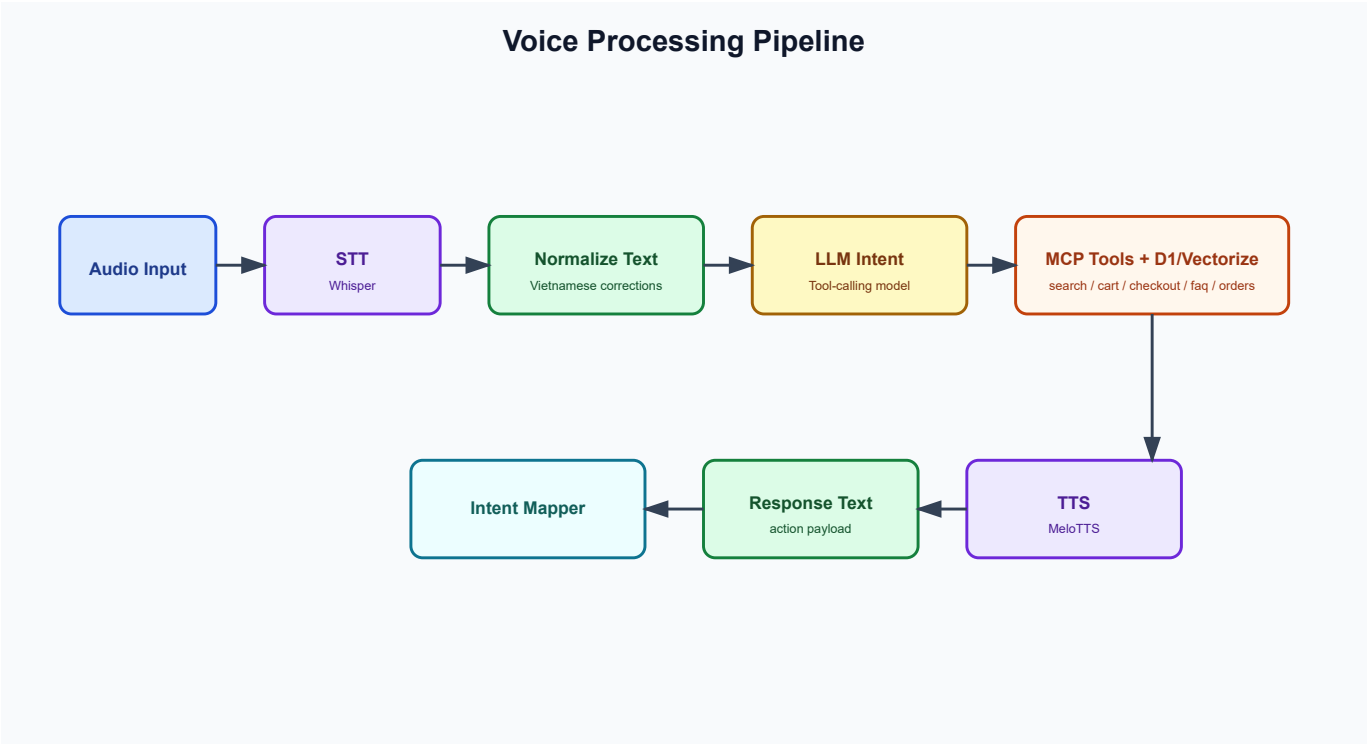

Fig. 6. Voice processing pipeline from audio input through STT, intent recognition, MCP tool execution, and TTS response.

MCP Tool Architecture: The implementation in `mcp.ts` defines transactional tools that execute commerce operations:

- `addToCart`: Adds product to cart with quantity
- `searchProducts`: Voice-aware product search
- `getOrderStatus`: Order tracking by order ID
- `confirmCheckout`: Voice-enabled checkout confirmation
- `createSupportTicket`: Voice-initiated support request

Why MCP Tools: The Model Context Protocol (MCP) provides structured tool definitions that LLM can reason about. Rather than generating free-form text responses, the voice pipeline can invoke typed tools with validated parameters. This approach reduces ambiguity in voice commerce scenarios where misinterpretation has financial consequences.

The AI Worker (`apps/ai-worker/src/index.ts`) implements:

- `/stt` using Whisper model.
- `/tts` using MeloTTS model.
- `/voice-process` orchestration pipeline: STT → LLM → MCP tools → intent post-processing.

The MCP layer (`apps/ai-worker/src/mcp.ts`) adds structured tool execution for search, filtering, compare, cart operations, checkout review/confirm, order status/cancel, FAQ retrieval, support ticket creation, review, address management, and promo validation ((Cloudflare, 2026d)).

Why this design: tool calls reduce hallucination risk versus free-form LLM responses by forcing transactional steps through explicit, typed tool interfaces.

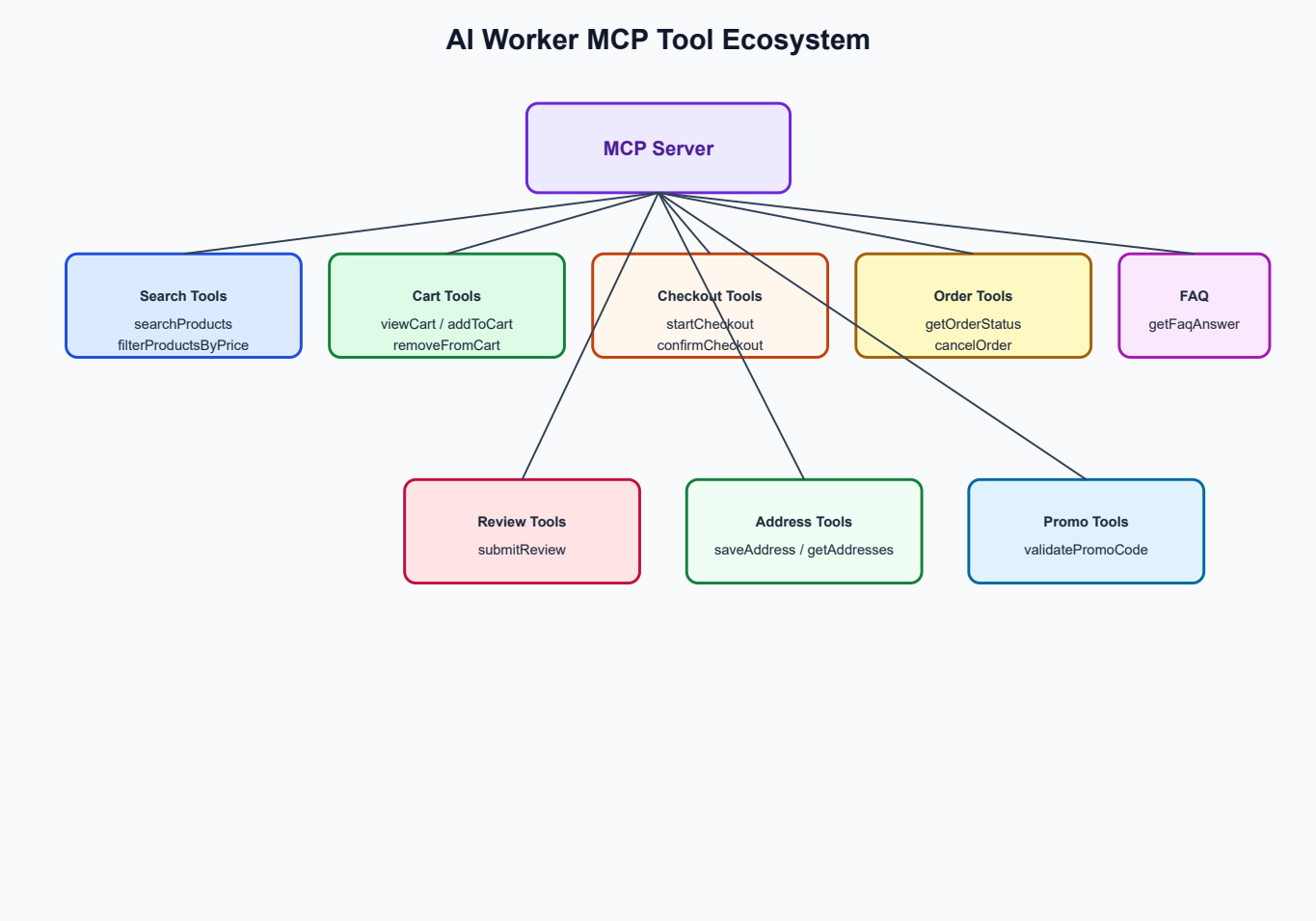

Fig. 7. MCP tool catalog used by AI Worker for commerce execution.

The crucial design detail is that the LLM is treated as a coordinator, not as a trusted source of business truth. By routing actions through MCP tools, each operation can enforce explicit input schema, database checks, and deterministic output shape before any frontend state mutation occurs. This directly supports safer voice-driven checkout and order management flows.

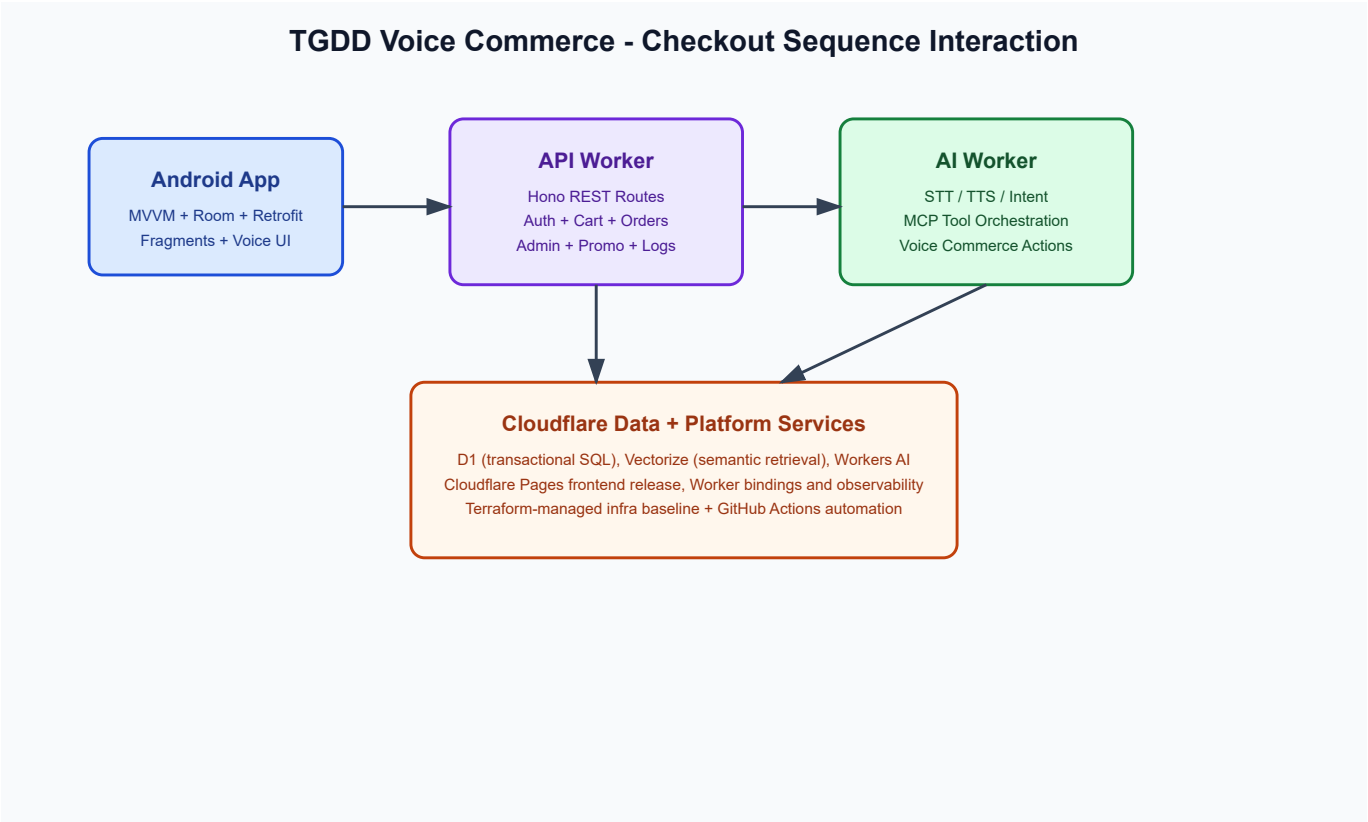

Fig. 8. Sequence-style interaction showing voice-assisted checkout confirmation path across Android app, AI worker, MCP tools, and API/D1 services.

This sequence perspective is important because it clarifies where failure recovery should occur. If STT quality degrades, normalization and intent fallback can recover before tool execution. If tool execution fails, the response still carries structured error messaging that the mobile layer can render predictably. If API/D1 persistence fails, checkout completion is not falsely confirmed, preserving transactional trust.

J. Endpoint-to-Screen Traceability (Main-Body View)

The table below maps key Android surfaces to API/AI execution paths. Keeping this mapping in the main body improves maintainability discussion because it shows how user journeys are implemented across boundaries.

Android Surface	User Action	Worker Endpoint/Tool	Why This Mapping Matters
ProductListFragment	Search by keyword/category	`GET /products` + search params	Keeps browse experience deterministic and cache-friendly
ProductDetailFragment	Open product details	`GET /products/:id`	Ensures detail view resolves canonical product data
CartFragment	Add/update/remove cart item	`/cart/*` routes	Separates optimistic local UX from server-authoritative quantity state
CheckoutFragment	Confirm checkout	`POST /orders` + validation	Prevents accidental success state before persistence
OrdersFragment	View order list	`GET /orders`	Supports post-purchase transparency and trust continuity
OrderDetailFragment	Inspect specific order	`GET /orders/:id`	Improves issue resolution and support context

Android Surface	User Action	Worker Endpoint/Tool	Why This Mapping Matters
Voice Panel	Voice search	`/voice-process` + `searchProducts`	Converts colloquial intent into bounded search operations
Voice Panel	Voice add-to-cart	`/voice-process` + `addToCart`	Guards transaction intent through typed tool inputs
Voice Panel	Voice checkout confirmation	`/voice-process` + `confirmCheckout`	Keeps high-impact actions explicitly structured
Voice Panel	Voice order status	`/voice-process` + `getOrderStatus`	Improves post-purchase accessibility without manual navigation
Voice Panel	Voice support ticket	`/voice-process` + `createSupportTicket`	Connects conversation to persistent support workflow

This mapping is intentionally journey-oriented, not endpoint-oriented. Users experience failures as broken journeys, not broken routes. By tracking route-to-screen coupling in this way, refactoring decisions can be prioritized by user impact and rollback risk.

K. Architecture-Level Failure Modes and Guards

Three recurring architecture-level failure modes were identified during integration:

1) **Shape drift**: route responses evolve without synchronized client/tool parsing updates.
2) **Authority drift**: local state and server state diverge after retries or transient failures.
3) **Intent drift**: natural-language references no longer map to stable product/order identifiers.

The implemented guards are layered accordingly:

- Shape drift guard: consistent response/error structure conventions and reusable route patterns.
- Authority drift guard: explicit sync methods with server-authoritative handling for financial state.
- Intent drift guard: normalization + typed tool schemas + disambiguation prompts.

These guards are important for long-term evolution because they reduce hidden coupling when features are added rapidly. They also make regressions easier to triage by indicating which class of invariant was violated.

V. Reflection on Assignment 2

Compared with an earlier prototype-style approach, this final iteration demonstrates stronger system integration depth across Android, API Worker, and AI Worker:

- Earlier-stage apps typically stop at UI + mock data; this submission integrates real auth, real cart/order persistence, and voice action execution against backend state.

The key improvement is process maturity: architecture decisions are now tied to maintainability and integration consequences, not just feature completion.

Another explicit evolution is confidence in cross-module integration. Earlier assignment-stage work often optimizes inside one layer (for example, polished screens without robust backend coupling). In this final project, decisions were made with integration-first thinking: mobile UX changes were checked against API behavior, and API shape changes were considered against voice tools. This shift from local optimization to system optimization is the most important learning outcome from a software evolution viewpoint.

The reflection also surfaces a practical communication lesson. As architecture became multi-layered, documenting boundaries became as important as writing code. Without explicit boundary notes (what Android handles locally, what API Worker validates and persists, and what AI Worker coordinates via tools), future maintainers could misunderstand ownership and introduce inconsistent fixes. Therefore, part of Assignment 3 progress was not only technical implementation but also technical narrative quality.

Assignment 2 feedback highlighted three gaps that directly informed this version. First, prior documentation described components but did not sufficiently justify architectural choices. In response, this report now pairs implementation claims with rationale (for example, why MVVM + Room + coroutines were selected for mobile-state safety, and why MCP tools were selected for transactional determinism). Second, previous testing discussion was generic; this version documents peer walkthrough evidence and links observed friction to concrete commits. Third, earlier process reporting was summarized at a high level; this version includes commit-level traceability in a detailed time log.

The most important personal learning was that integration quality is not achieved by finishing modules independently; it is achieved by validating cross-boundary behavior continuously. Several defects were not isolated bugs in a single layer, but boundary defects (for example, intent payload ambiguity between AI Worker and API contracts, or lifecycle continuity issues between Android UI state and repository synchronization). This changed the development strategy from "feature-complete per module" to "scenario-complete per user flow."

If repeated, one strategic change would be introducing earlier end-to-end checkpoints with fixed acceptance scenarios. In this project, major integration pressure clustered near the final days because multiple improvements landed in parallel. A future iteration would lock weekly scenario gates (e.g., browse→cart→checkout and auth→order tracking) so that integration risk is amortized throughout the schedule rather than compressed into final polish.

VI. Challenges, Explorations, and Takeaways

A. *Challenge: Balancing Offline-First UX with Server Truth*

Root cause. Mobile users need immediate UI responsiveness, but commerce actions must remain authoritative on server state.

Action taken. The Android repositories use local-first reads with network-aware fallback (`ProductRepository` and `CartRepository`) while synchronizing to server when online (`syncCart`, add/remove/quantity sync methods).

Takeaway. Local cache is essential for UX continuity, but each write path must include explicit resync strategy to avoid hidden divergence.

B. *Challenge: Making Voice Actions Deterministic*

Root cause. Natural language and STT noise create ambiguity in product references (for example ordinal phrases like "first item" vs explicit product names).

Action taken. The AI pipeline normalizes Vietnamese terms and maps conversation context into deterministic product IDs. MCP tools then execute typed operations (`addToCart`, `confirmCheckout`, `getOrderStatus`) instead of inferred text actions.

Takeaway. Voice commerce quality depends more on stateful intent grounding and tool contracts than on model size alone.

C. *Challenge: Cross-Platform State Synchronization*

Root cause. Mobile local state, server database, and voice conversation context can drift apart during flaky network conditions.

Action taken. The Android repositories implement explicit sync methods (`syncCart`, `syncOrders`) that compare timestamps and resolve conflicts using "latest wins" for non-financial data and server authority for financial data.

Takeaway. Optimistic local-first reads improve UX but require explicit write synchronization for data integrity, especially in commerce contexts where pricing and inventory affect transactional outcomes.

D. *Challenge: API Growth in a Monolithic Route File*

Root cause. Rapid feature addition into one `index.ts` improves speed initially but introduces cognitive and merge complexity.

Action taken. The current codebase mitigates risk with middleware guards and consistent error responses, but architectural debt is visible (route splitting and request validation should be next).

Takeaway. Delivery speed and long-term maintainability must be managed as an explicit trade-off, not an accidental outcome.

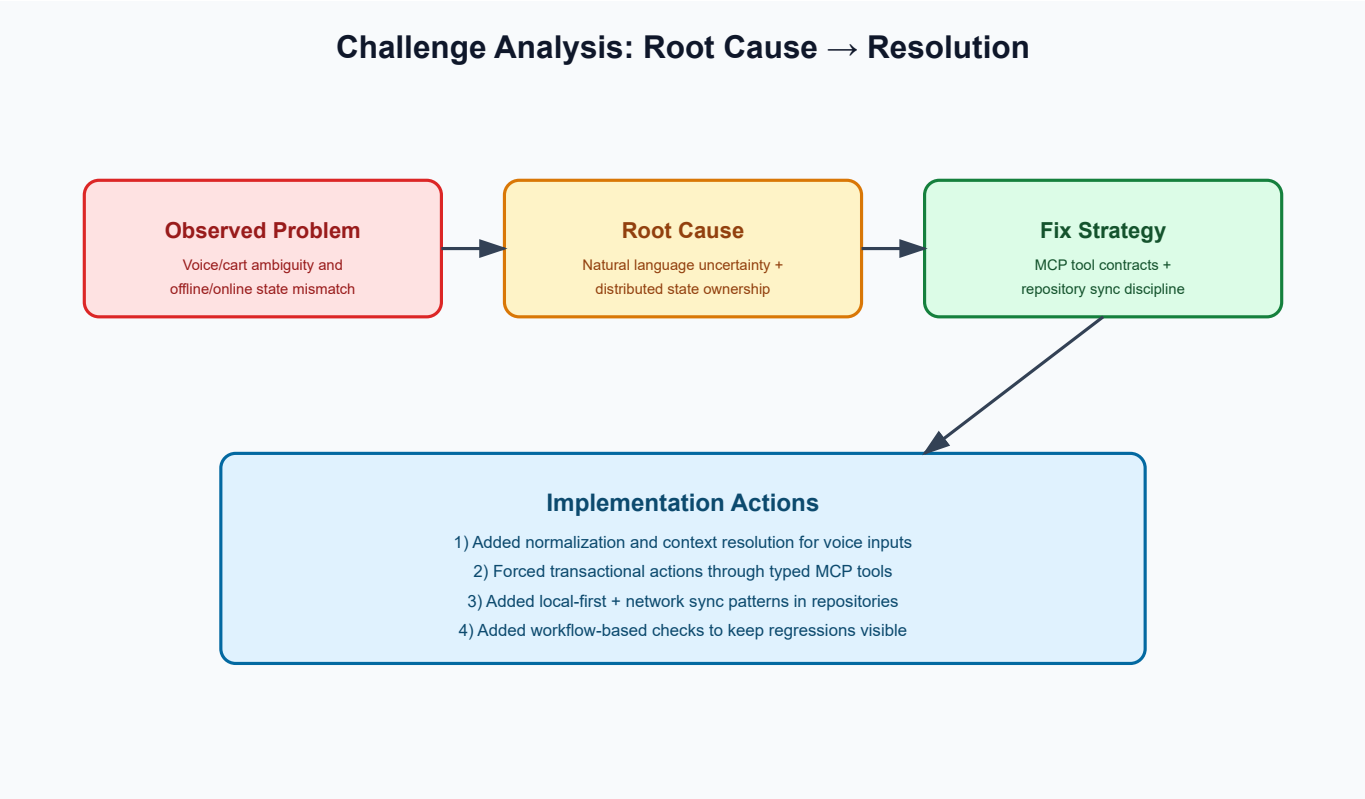

Fig. 9. Challenge-analysis model used in this report: observed issue, root cause, engineered mitigation, and downstream quality outcome.

E. Challenge: Maintaining Consistent User Intent Across Voice Turns

Root cause. Multi-turn conversations can lose context (for example references like "the second one"), causing incorrect product selection if prior search results are not preserved reliably.

Action taken. The AI worker context bundle carries recent search results and conversation history; intent mapping then resolves ordinal references into concrete product IDs before tool execution.

Takeaway. Conversational UX quality depends on memory design and context serialization as much as model output quality.

F. Challenge: Preserving API Contract Consistency During Rapid Feature Growth

Root cause. Frequent endpoint additions in a monolithic API file can cause inconsistent response shapes and validation behavior.

Action taken. The team maintained common guard/middleware patterns, standardized error responses, and reused route conventions across auth, cart, order, and support domains.

Takeaway. Rapid delivery remains sustainable only when contract consistency is enforced across routes, otherwise integration defects accumulate quickly in Android and AI Worker consumers.

G. Cross-Layer Validation Strategy (Applied During Challenges)

Across all challenges above, one repeatable strategy proved most effective: validate behavior as a *cross-layer contract* rather than as isolated module outputs. In practical terms, each critical scenario was validated through four checkpoints:

1) **Intent/Input checkpoint** — what the user asked (touch or voice) and how that request was normalized.
2) **Contract checkpoint** — whether API/tool payloads and response shapes were valid and deterministic.
3) **Persistence checkpoint** — whether canonical state changed correctly (cart/order/support records).
4) **Presentation checkpoint** — whether Android UI reflected the canonical result without stale or misleading states.

This method reduced false positives where one layer appeared successful while another silently failed. It also improved debugging speed because failures could be localized to a specific checkpoint before code changes were attempted.

From a software-evolution perspective, this strategy is reusable beyond this assignment. As systems grow, isolated unit success becomes less predictive of end-user reliability. Contract-sequenced validation helps teams preserve delivery speed without sacrificing correctness in high-impact flows such as checkout and order handling.

VII. Investigation / Experiment

This project includes a practical implementation experiment: "How far can tool-augmented voice flows go beyond FAQ bots in a transactional mobile commerce system?"

Hypothesis. A tool-centric AI worker can safely execute real shopping actions (search/cart/checkout/order status) with acceptable reliability.

Methodology.

- Implemented STT/TTS and LLM orchestration endpoints in AI Worker.
- Added MCP tool set for transactional operations against D1.
- Added intent post-processing in `intent.ts` to map tool results into frontend actions.
- Verified end-to-end path through API and mobile client integration points.

Results.

- The architecture supports true end-to-end voice-driven commerce actions rather than static response generation.
- Reliability was improved by deterministic tool routes and context-aware product resolution.
- Remaining gap: multilingual/misrecognition edge cases still require iterative tuning.

Conclusion. The experiment validates voice commerce as an execution layer on top of existing APIs, not a separate silo.

To strengthen future experimental rigor, the next iteration should add explicit benchmark instrumentation for at least three dimensions:

- STT-to-action latency under variable network conditions.
- Intent-to-tool precision on colloquial Vietnamese phrasing.
- Checkout completion rate difference between purely manual flow and voice-assisted flow.

Even without those full quantitative datasets in this submission, the current implementation already provides structural hooks for such evaluation because tool calls and route boundaries are explicit and loggable.

An important secondary finding from this experiment is that voice UX should be evaluated on **task completion stability** rather than only transcription quality.

A. Voice Commerce Performance Observations

The implemented voice pipeline in `apps/ai-worker/src/` provides structural hooks for performance evaluation:

STT Processing:

- Model: `@cf/openai/whisper-large-v3-turbo` (Cloudflare Workers AI)
- Input: Audio base64 encoded in request body

- Output: Transcribed text passed to intent processing

TTS Processing:
- Model: `@cf/myshell-ai/melotts` (Cloudflare Workers AI)
- Input: Text from intent response or direct endpoint
- Output: Audio base64 for client playback

Pipeline Latency Variables Consider:
1) Audio upload size (varies by encoding quality)
2) STT model inference time (edge-dependent)
3) Intent processing LLM call (network-dependent)
4) Tool execution time (D1 query latency)
5) TTS model inference (edge-dependent)
6) Audio download size

The implementation includes explicit logging for each pipeline stage in `/voice-process` route, enabling latency analysis after integration updates. A production monitoring system would aggregate these logs to calculate P50/P95/P99 latency percentiles per pipeline stage.

Intent Precision Considerations: Vietnamese language processing introduces specific challenges:
- Tone marks affect meaning (á, à,ả, ã, ạ)
- Colloquial abbreviations common in commerce ("sp" for "sản phẩm", "giá" for "giá tiền")
- Regional pronunciation variations
- Background noise from mobile environments

The intent.ts implementation addresses these through:
- Text normalization before LLM processing
- Fuzzy product matching using search API
- Confirmation prompts for ambiguous references
- Tool schema constraints preventing invalid actions

A perfectly transcribed command can still fail user expectations if intent mapping or data-layer execution is ambiguous. Conversely, mildly noisy transcription can still produce a successful outcome when context grounding and tool constraints are robust. This indicates that end-to-end system evaluation should prioritize operational correctness and user trust continuity over isolated model metrics.

B. Evaluation Framework for Future Iterations

To make future evaluations comparable over time, the project can adopt a fixed evaluation framework with explicit thresholds and failure classes.

Recommended core metrics:

- **Voice-to-action latency:** P50/P95 across STT, intent, tool execution, and response synthesis stages.
- **Action correctness rate:** percentage of sessions where intended operation matches executed operation.
- **Recovery success rate:** percentage of failed attempts that recover without user abandoning the flow.
- **Checkout trust continuity:** percentage of sessions where users report confidence in transaction state after confirmation.

Recommended failure taxonomy:

- **Type A (Interpretation):** STT/intent ambiguity before tool dispatch.
- **Type B (Contract):** invalid or mismatched payload/response shapes.
- **Type C (Persistence):** canonical state not updated as expected.
- **Type D (Presentation):** Android UI not synchronized with canonical result.

Using this taxonomy would improve evolution quality because each failure class maps naturally to a responsible layer and remediation strategy. It also supports cleaner retrospective analysis by separating model-quality issues from platform-contract and UI-state issues.

C. Scenario-Based Success Criteria

To prevent metric-only optimization, each critical journey should also have scenario-level success criteria:

- **Browse-to-cart scenario:** user can discover an item, inspect details, add to cart, and confirm cart state in one uninterrupted loop.
- **Auth-to-order scenario:** user can authenticate, proceed through checkout, and retrieve order status without contract mismatch.
- **Voice-assisted scenario:** user can execute a voice search/action and receive deterministic, UI-renderable outcomes with fallback paths.

A scenario is considered successful only if all four cross-layer checkpoints pass (intent, contract, persistence, presentation). This criterion is stricter than single-endpoint success but better aligned with real user outcomes.

Applying scenario-level gates during development would reduce late-stage integration surprises and make release readiness decisions more objective.

D. Experimental Validity and Limitations

The experiment demonstrates architectural feasibility, but it is important to state validity limits. The current runs were integration-oriented and scenario-driven, not a large-sample controlled experiment. Therefore, conclusions should be interpreted as engineering validation (the approach works and is testable) rather than statistical proof of population-level UX superiority.

Potential confounders include network variability, microphone quality, and user phrasing style. These factors can dominate perceived latency and command success. To reduce confounding in future work, evaluation should standardize input audio profiles, classify background-noise bands, and separate model-inference latency from transport latency.

Despite these limits, the experiment still produces useful evidence for software evolution: the boundaries are explicit, failures are observable, and behavior is recoverable through deterministic tool calls. This is sufficient to justify the architecture as a practical baseline for iterative improvement.

VIII. Software Development Process

A. Version Control and Integration Strategy

Development was tracked as iterative integration across three modules (Android, API Worker, AI Worker). Commits were grouped by user-facing flow changes (auth, cart, checkout, profile/help, voice interaction) to keep traceability between report claims and concrete implementation updates.

Commit semantics were treated as process artifacts, not only version-control mechanics. Feature commits (`feat(...)`) were used to indicate user-visible capability additions, fix commits (`fix(...)`) captured behavior corrections, refactor/chore commits captured structural and maintainability changes, and merge commits marked integration boundaries. This convention made it easier to reconstruct the implementation story for reflection and risk review.

Integration sequencing followed a flow-first strategy:

- Stabilize Android interaction and persistence behavior.
- Stabilize API Worker transactional contracts.
- Layer AI Worker orchestration on top of existing contracts.
- Re-verify end-to-end scenarios after each major update cluster.

This sequencing reduced the probability that voice behavior would mask unresolved commerce-contract issues. It also improved debugging speed because failures could be localized by layer before cross-layer tuning.

B. Time Log

The table below lists every commit from `android/` history.

Date	Commit	Message
2026-04-12	8a50192	feat; add more product
2026-04-12	cb3fb52	feat: add more feature
2026-04-12	4318778	fix: migrate all to string.xml
2026-04-12	c5b4d01	feat: voice ai
2026-04-12	201238b	feat(nav): update navigation graph
2026-04-12	52731a2	feat(res): update strings and themes
2026-04-12	ae0f216	feat(app): add voice assistant and localization support
2026-04-12	df32e37	build: update build.gradle.kts
2026-04-12	77e503a	feat(ui): update help layout
2026-04-12	cb12375	feat(ui): update product list layouts
2026-04-12	c9b3021	feat(ui): update cart and checkout layouts
2026-04-12	bf5bf78	feat(ui): update profile and orders layout files
2026-04-12	7563cfc	feat(ui): update auth layout files
2026-04-12	ae17cf7	feat(app): update MainActivity
2026-04-12	da65492	feat(network): update NetworkModule
2026-04-12	7a75955	fix(data): update ProductDto model
2026-04-12	53b784e	feat(help): update HelpFragment
2026-04-12	76366f5	feat(cart): update CartFragment
2026-04-12	07777c3	feat(detail): update product detail fragment and ViewModel
2026-04-12	ed431d9	feat(product): update product list fragment and ViewModel
2026-04-12	d94c48a	feat(orders): update OrdersViewModel
2026-04-12	6b7c368	feat(orders): update OrdersFragment UI
2026-04-12	65d73cf	feat(profile): update ProfileViewModel
2026-04-12	8b17a7e	feat(profile): update ProfileFragment UI
2026-04-12	f05022b	feat(auth): update AuthViewModel and VerifyEmailViewModel
2026-04-12	ca871c4	feat(auth): update login/register/reset-password/verify-email UI fragments
2026-04-12	668ca2d	fix: cart and auth
2026-04-11	037e9df	feat(firebase/oauth): integrated android
2026-04-10	82a8b9c	feat: implement android features
2026-04-09	95681c2	fix: test all pass
2026-04-09	9d4b376	repo maintenance: integration fix
2026-04-03	51fb27d	feat: add payments
2026-03-29	3a791d4	fix: comprehensive bug fixes across Android app
2026-03-29	e53a680	fix: remove Gson LOWER_CASE_WITH_UNDERSCORES naming policy

Date	Commit	Message
2026-03-29	47b45d2	refactor: add gitignore
2026-03-29	24ff07e	chore: add Android .gitignore
2026-03-29	e8f7cda	feat: apply UI/UX Pro Max improvements
2026-03-22	5c780ed	chore: push lock files
2026-03-17	cea7f8e	feat: enhance ai search power
2026-03-16	569ae85	security workflow update
2026-03-16	e3d78e6	chore: remove temp build files
2026-03-16	1d8b192	worker configuration fix

Table 1. Full Android commit log used for process evidence.

Time-management reflection. The full commit-level log shows strong delivery velocity but also a visible end-loaded spike on 2026-04-12. Future iterations should spread integration and UI cleanup into earlier checkpoints to reduce last-day compression risk.

Because this report is generated from repository evidence rather than a personal daily diary, the timeline below summarizes cross-module phases that each include Android, API Worker, and AI Worker outcomes:

- Phase 1 — Foundation alignment: Android navigation/state scaffolding, API Worker baseline commerce/auth endpoints, AI Worker STT/TTS pipeline setup.
- Phase 2 — Commerce execution path: Android product/cart/checkout screens, API Worker cart/order transaction routes, AI Worker intent-to-tool mapping for shopping tasks.
- Phase 3 — Account and trust flows: Android login/register/verify/profile UX, API Worker session and verification behavior, AI Worker response shaping for authenticated actions.
- Phase 4 — Experience refinement: Android voice/localization and UI polish, API Worker contract consistency hardening, AI Worker tool orchestration and conversational stability tuning.

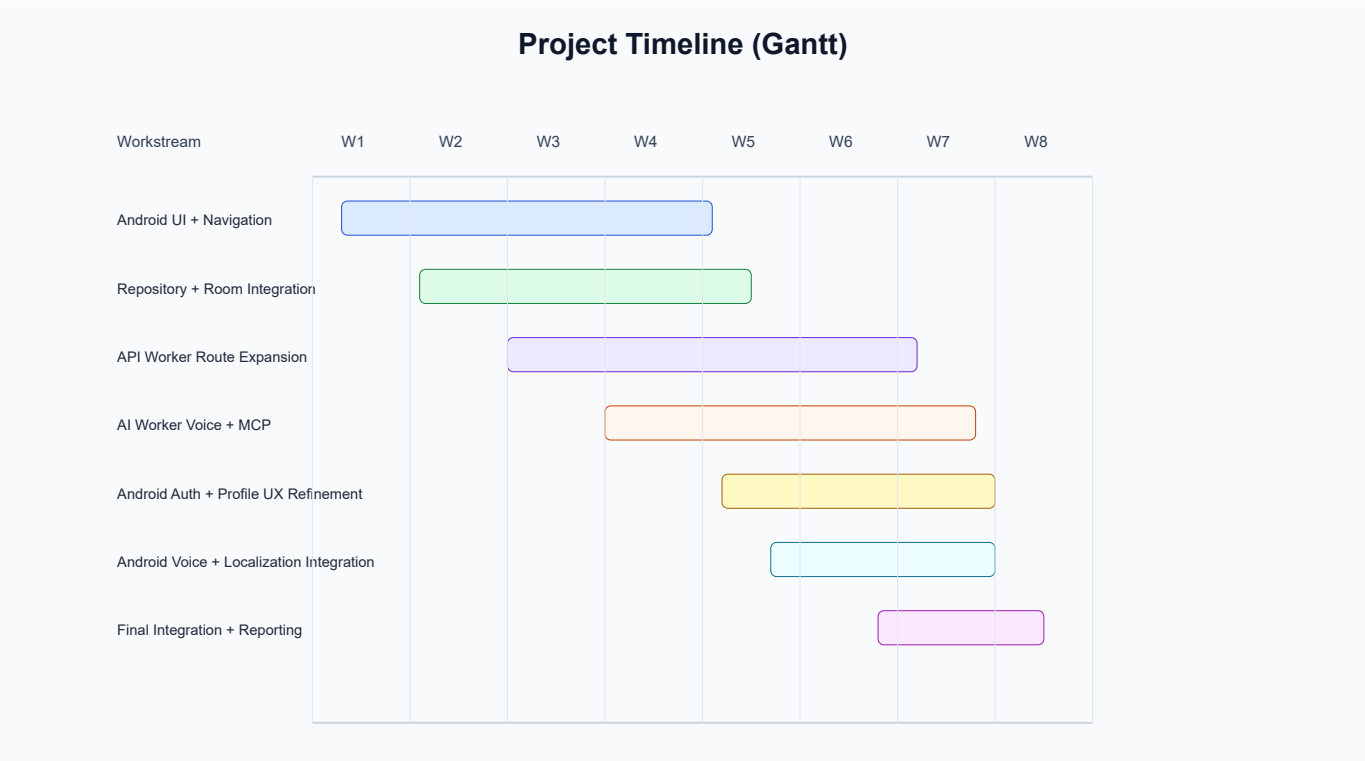

Fig. 10. Artifact-based timeline for major implementation streams.

The timeline also reveals concurrency patterns in team execution. Mobile and backend streams progressed in overlapping windows, while AI orchestration and integration hardening were layered later as integration risk became more visible. This sequencing is a reasonable strategy for time-boxed delivery: establish baseline transactional correctness first, then add interaction sophistication, then stabilize cross-module behavior.

A future improvement would be attaching effort points or hour ranges to each stream so that schedule variance can be analyzed quantitatively instead of qualitatively. Even simple effort tagging would enable better sprint calibration for subsequent iterations.

IX. Generative AI Acknowledgement

No Generative AI tools were used for this task

X. Appendices

A. *Appendix A: Architecture Notes*

- Android uses MVVM + Hilt + Room + Retrofit and fragment-based navigation.
- API Worker centralizes commerce/auth endpoints in Hono with D1-backed transactional data access.
- AI Worker provides STT/TTS + LLM orchestration + MCP transactional tools with Vectorize/D1-backed retrieval and action grounding.

Architecture evidence mapping for rubric alignment:

- Use-case perspective: actor-centered journeys are represented by the navigation topology and cross-module sequence figures.
- Activity-flow perspective: checkout and voice-processing paths are represented by sequence and pipeline diagrams.

References

Android Developers (2026c) *Kotlin Coroutines on Android.* Available at: https://developer.android.google.cn/kotlin/coroutines.

Android Developers (2026b) *Save Data in a Local Database Using Room.* Available at: https://developer.android.google.cn/training/data-storage/room.

Android Developers (2026a) *ViewModel Overview.* Available at: https://developer.android.google.cn/topic/libraries/architecture/viewmodel.

Cloudflare (2026a) *Cloudflare D1 Documentation.* Available at: https://developers.cloudflare.com/d1/.

Cloudflare (2026d) *Cloudflare Vectorize Documentation.* Available at: https://developers.cloudflare.com/vectorize/.

Cloudflare (2026c) *Cloudflare Workers AI Documentation.* Available at: https://developers.cloudflare.com/workers-ai/.

Cloudflare (2026b) *Cloudflare Workers Documentation.* Available at: https://developers.cloudflare.com/workers/.